VAASAN AMMATTIKORKEAKOULU
UNIVERSITY OF APPLIED SCIENCES

Nguyen An Phuong Linh

PERFORMANCE ANALYSIS OF ENSFM RECOMMENDATION SYSTEMS ON MODERN E-COMMERCE DATASETS

Technology
2025

VAASAN AMMATTIKORKEAKOULU
Bachelor of Engineering

ABSTRACT

Author	Nguyen An Phuong Linh
Title of the thesis	Performance Analysis of ENSFM Recommendation System on Modern E-commerce Datasets
Year	2025
Language	English
Number of pages	62 + 8 appendices
Supervisor	Harri Lehtinen

Recommender systems are essential tools that analyze user b ehaviors, preferences, or characteristics to provide tailored suggestions. Various recommender methods exist, such as collaborative filtering, content-based filtering, hybrid models, and context-aware techniques. However, efficiently deploying any of these on extremely large-scale datasets remains a major challenge. This thesis addresses the research gap by investigating the application of Efficient Non-Sampling Factorization Machines (ENSFM) to large-scale, context-rich datasets. Specifically, the thesis evaluates the feasibility of training the ENSFM model using real-world datasets that are significantly larger and more complex than the original benchmark datasets.

The research primarily utilizes Efficient Non-Sampling Factorization Machines (ENSFM), an advanced factorization model optimized to handle large-scale implicit datasets effectively without relying on negative sampling. Experiments were conducted using two real-world datasets—Yelp2018 and Amazon Book Reviews 2023—after rigorous preprocessing steps, including data cleaning, reindexing of user and item IDs, and leave-one-out splitting for training and evaluation purposes.

The experimental results indicate that the ENSFM model demonstrates limited effectiveness when applied to multi-category datasets such as Yelp2018 and Amazon Book Reviews 2023. Specifically, the model struggles with accuracy issues, long computational times, high computational costs, demanding hardware requirements, and poor scalability. These limitations highlight the necessity of further optimization and adaptation of ENSFM for handling complex, multi-category real-world datasets.

Keywords recommender systems, context-aware, model evaluation, real-world datasets

TABLE OF CONTENTS

ABSTRACT 2
1 INTRODUCTION 7
1.1 Background and Perspectives of the Thesis Topic 7
1.2 Thesis Objectives, Research Questions 7
1.3 Tasks and Methods 8
1.4 Thesis Structure 9
1.5 Use of AI in This Thesis 10
2 KNOWLEDGE BASE AND THEORY 11
2.1 Introduction to Recommender System 12
2.1.1 General Concept of Recommender System and Implicit Feedback 12
2.1.2 Context-aware Recommendations 13
2.2 Top-K Recommendations 14
2.3 Negative Sampling vs. Non-sampling Technique 14
2.3.1 Negative Sampling Technique 14
2.3.2 Non-sampling Technique 15
2.4 Foundation Model 16
2.4.1 Factorization Machines 16
2.4.2 Efficiency Matrix Factorization without Sampling 17
2.5 Evaluation Methodologies 18
2.5.1 Leave-one-out Evaluation Protocol 18
2.5.2 Hit Ratio (HR) 18
2.5.3 Normalized Discounted Cumulative Gain (NDCG) 19
2.6 Efficient Non-Sampling Factorization Machines (ENSFM) 20
2.6.1 Overview 20
2.6.2 ENSFM Learning Algorithm 22
3 IMPLEMENTATION 24
3.1 Technology 24
3.1.1 Python 24
3.1.2 Jupyter Notebook 24
3.1.3 NumPy 25

3.1.4 Pandas 25
3.1.5 Matplotlib 26
3.1.6 TensorFlow 26
3.1.7 Google Colab and Kaggle 27
3.2 Datasets 27
3.2.1 Original Datasets Information 28
3.2.2 Single and Multi-category Datasets 33
3.3 Testing methodology 36
3.4 Procedures 37
3.4.1 System Version Upgrade 37
3.4.2 Re-experimentation 37
3.4.3 Dataset Selection 38
3.4.4 Data Preprocessing 38
3.4.5 Model Training and Evaluation 47
4 RESULTS 49
4.1 ENSFM Model Results on the New Dataset (Research Question 1) 49
4.2 Key Factors Affecting the ENSFM Model's Performance on the New Dataset (Research Question 2) 53
5 SUMMARY AND REFLECTION 55
REFERENCES 57
APPENDICES 63
Appendix 1. Statistical Overview of All the Datasets. 63
Appendix 2. Extract Unique JSON Keys from Yelp2018 Dataset. .. 65
Appendix 3. Extract Unique JSONL Keys from Amazon Books Review 2023 Dataset. 67
Appendix 4. Listing the Book Category Distribution in the Amazon Books Reviews 2023 dataset. 69

LIST OF FIGURES

Figure 1. ENSFM Framework Architecture (Chen et al. 2020) 21
Figure 2. ENSFM Learning Algorithm (Chen et al. (2020)), which describes how the model is trained using a non-sampling strategy combined with mini-batch updates. 23
Figure 3. Unique keys of Yelp2018 dataset - `business.json` file 30
Figure 4. Unique keys of Yelp2018 dataset - `review.json` file 30
Figure 5. Unique keys of Yelp2018 dataset - `user.json` file 31
Figure 6. Unique keys of Yelp2018 dataset - `checkin.json` file..... 31
Figure 7. Unique keys of Yelp2018 dataset - `tip.json` file 32
Figure 8. Unique keys of Amazon Books Reviews 2023 dataset - `Books.jsonl` file 33
Figure 9. Unique keys of Amazon Books Reviews 2023 dataset – `meta_Books.jsonl` file 33
Figure 10. Most Common Business Categories list of Yelp2018 dataset 35
Figure 11. An example of feature input for each user and item (Chen et al. 2020, p.2) 41
Figure 12. General preprocessing steps for this thesis datasets..... 41
Figure 13. Performance of the ENSFM model on all the datasets 51
Figure 14. Comparison of training time of each dataset 52

LIST OF TABLES

Table 1. An overview of the notation and its symbols 11
Table 2. Source Datasets Statistics 28
Table 3. Statistical details of the single and multiple categories dataset that relate to Yelp2018 dataset 33
Table 4. Statistical details of the single and multiple categories dataset that related to Amazon Books Reviews 2023 dataset..... 36
Table 5. Evaluation Results 49

Table 6. Statistical details of the training time of every dataset on GPU Tesla P100-PCIE-16GB 52
Table 7. Statistics of datasets in this thesis 63

ABBREVIATIONS

ISBN	International Standard Book Number
BPR	Bayesian Personalized Ranking
CARS	Context-Aware Recommender Systems
FM	Factorization Machines
HR	Hit Ratio
NDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
ENSFM	Efficient Non-Sampling Factorization Machines

1 INTRODUCTION

By analyzing user behaviors, interests, or other characteristics, recommender systems are designed to identify and present suitable suggestions. These systems are normally applied in e-commerce, online services, social media, and other platforms to help users discover products, content, or services that they might be interested in. Recommender systems are indispensable parts of e-commerce companies, where revenue depends on selling products and services on websites or in applications like Shopee, Amazon, Taobao, Shein, or with companies that offer a variety of products, such as YouTube, Spotify, Netflix, and others.

1.1 Background and Perspectives of the Thesis Topic

Currently, various types of recommender systems have been developed such as collaborative filtering, content-based filtering, hybrid recommendation and context-aware recommendation. However, the deployment of these recommendation models on large datasets containing numerous outliers and requiring substantial computational resources has rarely been addressed thoroughly. This thesis is motivated by a research gap and aims to explore the application of large-scale datasets in training Efficient Non-Sampling Factorization Machines (ENSFM) with the specific goal of optimizing performance in context-aware recommendation tasks.

1.2 Thesis Objectives, Research Questions

This study mainly wants to check if using new, big data sets works well with an existing recommendation system – specifically, ENSFM (Efficient Non-Sampling Factorization Machines). The study aims to evaluate how well the ENSFM model performs when trained on real-world datasets

that are significantly larger and more complex than the original benchmark datasets used by the authors.

To achieve this, the thesis will conduct a comparative analysis between the model's performance on the newly applied datasets and its performance on the original datasets. In addition, a series of controlled experiments will be designed to identify and examine the key factors within the new datasets that significantly influence the model's effectiveness. Through this dual approach - performance evaluation and factor analysis, the thesis seeks to provide a deeper, more practical understanding of deploying efficient recommendation models in real-world, large-scale data environments.

It should be noted that the main goal of this thesis is not to try to achieve the best performance but to test the feasibility of applying the ENSFM model to a new, larger, and more complex dataset. The complete source code is available on Github.[1]

This thesis aims to answer two main research questions, which are directly informed by the objectives outlined previously:

- Question 1: Does the ENSFM model work well on the new dataset?
- Question 2: Which factors in the new datasets strongly influence the model's performance?

1.3 Tasks and Methods

Because this thesis topic focuses on applying new datasets to an existing recommendation model, the first step is selecting suitable, real-world datasets. After choosing a dataset, preprocessing is an important step.

[1] https://github.com/Freya-Ng/Performance-Analysis-of-ENSFM-Recommendation-Systems-on-Modern-E-Commerce-Datasets

We removed unnecessary data such as items' pictures, videos, users' friend lists, and outlier data. Once the data was cleaned, user IDs, item IDs were reindexed into new sequential formats so that they would be easier for the model to learn contextual information.

Using the leave-one-out methodology, the dataset was then divided into distinct training and test portions. The details of these steps will be discussed in the following sections. In addition to evaluating model performance, we also investigate which factors within the dataset and training process most significantly influence the model's learning outcomes.

1.4 Thesis Structure

Chapter 1 introduces the background, motivations, and research questions of the thesis. Chapter 2 provides the theoretical foundation of recommender systems, covering key concepts such as implicit feedback, context-aware recommendation, Top-K ranking, and evaluation metrics. It also presents the core model used in this study - Efficient Non-Sampling Factorization Machines (ENSFM) - and its underlying learning mechanism.

Chapter 3 discusses the implementation process, including the tools and technologies employed (such as Python, TensorFlow, and Google Colab), and details the selected datasets. It explains the end-to-end workflow from data selection, preprocessing, encoding, and reindexing to the training/testing split using the leave-one-out strategy. The chapter also outlines the experimental setup and reimplementation procedure used to adapt ENSFM to new large-scale datasets.

Chapter 4 reports the experimental results and analysis, contrasting the model's performance between the original and new datasets. It also identifies influential factors within the data that affect learning out-

comes. Finally, Chapter 5 summarizes the thesis contributions, discusses the limitations, and provides recommendations for future research directions.

1.5 Use of AI in This Thesis

In this thesis, I have utilized ChatGPT (versions such as 4.0-mini, 4.5, etc.) to refine the language, correct grammatical errors, improve word choice, and enhance the overall logical flow of my writing. The primary application of AI occurred during the writing and editing phases, where ChatGPT assisted in making the text clearer and more coherent while preserving my original ideas and intentions.

For language enhancement, I employed Grammarly's AI tool to check for plagiarism in small sections of the report, ensuring the originality and integrity of the content. Additionally, ChatGPT was helpful in retrieving basic information and assisting with quick citations. I cross-verified all AI-generated information through reliable sources such as Google Search to ensure the accuracy and credibility of the content.

I have ensured the authenticity of my work by verifying all external information and referencing it appropriately. AI was not employed to generate novel research findings or conclusions but rather to assist with structuring content, refining text, and streamlining the citation process. Throughout the process, I prioritized data protection and academic integrity by ensuring no sensitive or proprietary data was input into AI tools.

All AI applications were used responsibly, with appropriate oversight, to maintain the integrity and originality of my work.

2 KNOWLEDGE BASE AND THEORY

To enhance clarity and consistency throughout the thesis, Table 1 summarizes the notations and symbols used in the modeling and evaluation processes. These notations are applied in the description of factorization models, learning algorithms, and loss functions presented in the subsequent chapters. We refer to this table when interpreting mathematical expressions or implementation details.

Table 1. An overview of the notation and its symbols

Symbol	Definition
$\mathcal{U}$	Set of users
$\mathcal{V}$	Set of items
$\mathcal{B}$	Group of users
$\mathcal{X}$	Features group
$\mathcal{Y}$	User-item interactions
$\mathcal{R}$	Collection of user-item pairings with non-zero values
x	Sparse input data
d	Count of latent factors
m	Quantity of user context features
n	Quantity of item context features
w_0	Global bias
w_i	First-order feature interaction weight
e_i	The feature i's latent vector

h	Weights of neurons in the prediction layer
$\widehat{y_{uv}}$	Predicted score for user u and item v
y_{uv}	Ground truth interaction between user u and item v
c_{uv}	Weight of entry y_{uv}
p_u	User context auxiliary vector u
q_v	Item context auxiliary vector v
h_{aux}	Weights of the prediction layer's auxiliary neurons
Θ	Set of neural parameters
$\circ$	Element-wise (Hadamard) product
$\langle \cdot, \cdot \rangle$	Inner product between two vectors

2.1 Introduction to Recommender System

2.1.1 General Concept of Recommender System and Implicit Feedback

Recommender systems suggest personalized recommendations on items to users based on various kinds of information on users and items according to Pilászy et al. (2010). It is a system designed to help users navigate and search within a large set of items (for example: products, movies, songs, …) by providing personalized recommendations for each user. In particular, the system will filter or organize the objects based on information like user profiles, item features, and/or habits/behaviors (purchases, views, ratings, reviews, …) of many users. The results are lists or groups of the most suitable items for each user's interests and

demands. This personalization is the difference between the recommender system and a normal most popular list.

To improve personalization, in addition to explicit behaviors (such as star ratings or reviews), another important aspect is implicit feedback. This refers to feedback collected from user actions and interaction traces rather than through a rating form. Examples of implicit feedback range from search patterns and browse activity to purchase history and even mouse movements, according to Hu et al. (2008). Analyzing these indirect signals helps the system capture the user's real preferences, thereby enhancing suggestion quality even if the user rarely (or never) provides explicit feedback.

2.1.2 Context-aware Recommendations

By primarily relying on the relationship between users and items (user-item), conventional recommender systems frequently neglect contextual elements that can significantly influence a user's decisions. Meanwhile, for the purpose of providing personalized item suggestions to users, CARS leverage contextual data such as user profile information, item features, and details regarding the transaction's timing and location like in these studies Chen et al. (2020), Santana & Domingues (2020), Bayer et al. (2017), Rendle et al. (2011) and Yuan et al. (2016).

Three primary groups encompass the different approaches within CARS: pre-filtering, post-filtering, and contextual modeling. Among these, contextual modeling is considered the most effective when context is treated as an integral part of the recommender model. By modeling interactions between users, items, and contexts, CARS can learn complex and suitable behavioral patterns that reflect real-world scenarios. This is especially important in domains such as e-commerce, tourism, and healthcare, where customers' decisions are often strongly affected by contextual background factors.

2.2 Top-K Recommendations

In recommender systems, Top-K recommendation is the method by which the system filters out a list of the K most suitable products or services to introduce to users. Instead of viewing hundreds of different products, users will immediately see a short list of the most potential items, which have been ranked according to their suitability to users' interests and demands.

More specifically, to generate this list, the system evaluates users' interest and interaction level with each product based on their previous interaction history (implicit feedback). Because users do not often rate products explicitly, the system relies on these indirect signals to infer their preferences. From that, it calculates a ranking score for each product and then chooses the items with the highest ranking scores.

In this thesis project, we use the same evaluation method as the authors of the paper "Efficient Non-Sampling Factorization Machines for Optimal Context-Aware Recommendation" without changing the K-values according to Chen et al. (2020). In their experiments assessing different recommendation lengths, Chen et al. (2020) employed values of 5, 10, and 20 for the length parameter K. These decisions are highly business-specific and thus should be made based on the context and the field of the business in question, according to Le (2019).

Using Top-K recommendations not only saves users' searching time but also improves their shopping experience and interaction, thereby bringing significant benefits to both businesses and customers.

2.3 Negative Sampling vs. Non-sampling Technique

2.3.1 Negative Sampling Technique

Negative sampling is a common and efficient technique for training models when working with implicit feedback data, characterized by having

only observed positive interactions between users and items. Different from explicit feedback data – where users give explicit scores – in implicit feedback data, missing interactions does not mean that it is similar to the users not liking those items. Consequently, the model training process requires a strategy for handling negative cases. For the BPR learning algorithm specifically, negative sampling addresses this by selecting items that a user has not rated. Moreschini (2021) describes this task as choosing these unobserved interactions (assumed to be negative) to optimize the model parameters.

Instead of pretending all of the unobserved interactions are negative (a simple method but extremely computationally expensive due to the large number), the negative sampling technique will select a small subset from unobserved interactions and consider them as negative examples. These examples are fed into the training process to help the model learn to distinguish between relevant and irrelevant products for each particular user.

This method saves significant computational resources, especially when working with large datasets containing millions of users and items. However, the performance of the model can depend on the negative sampling strategy used – for example, random sampling, popularity-based sampling, or hard negative sampling, according to Ma et al. (2024). Choosing the right sampling strategy will directly affect the quality of the final recommendation model.

2.3.2 Non-sampling Technique

One of the biggest disadvantages of the recommender system using implicit feedback is the dependence on negative sampling technique. Although this technique helps reduce computing expenses but easily skips many important information and causes noises because of the randomness in the sampling process.

In the study by Chen et al. (2020), the authors proposed an alternative approach is non-sampling. In that research, all of the unobserved interactions are fed into the training process as negative signals. In order to handle the expensive computing cost, Chen et al. designed the Efficient Non-Sampling Factorization Machines (ENSFM) model – an efficient machine learning framework for context-aware recommendation tasks. ENSFM not only ensures the ability to learn from the entire data but also maintains a fast training speed thanks to the memorization strategies and model structure reorganization. Experimental results show that ENSFM not only outperforms traditional sampling models in terms of accuracy but also trains significantly faster.

2.4 Foundation Model

2.4.1 Factorization Machines

Factorization Machines (FM) is a flexible and powerful framework that combines the ability of feature engineering with the predictive strength of latent factor models. FM uses latent embeddings to estimate the target from a real-valued input vector x. The prediction function is defined as:

$$\widehat{y_{FM}}(x) = w_0 + \sum_{i=1}^{m+n} w_i x_i + \sum_{i=1}^{m+n} \sum_{j=i+1}^{m+n} e_i^T e_j \cdot x_i x_j \quad (1)$$

In this equation:

- w_0 is the global bias term,
- w_i represents the weight for the i-th feature,
- $\boldsymbol{e_i} \in \mathbf{R^d}$ is the latent vector that captures interactions with other features,
- The term $\boldsymbol{e_i^T e_j}$ the term denotes the factorized interaction.

Despite FM having linear time complexity per instance, training without negative sampling remains a challenge. In implicit feedback settings,

observed interactions are sparse, while unobserved ones dominate the data. A widely used non-sampling loss is:

$$L(\Theta) = \sum_{u \in \mathcal{U}} \sum_{v \in \mathcal{V}} c_{uv} (y_{uv} - \widehat{y_{uv}})^2 \tag{2}$$

Where c_{uv} denotes the weight of the entry y_{uv}. In practice, y_{uv} is 1 for observed interactions and 0 otherwise, but all entries are still weighted by c_{uv} to reflect confidence in the data.

While FM is efficient per instance, the overall complexity of computing this full loss is prohibitive: $\mathcal{O}\big((m+n)|U||V|d\big)$.Given that $|\mathcal{U}||\mathcal{V}|$ can reach billions in real-world applications, this cost becomes intractable for large-scale recommendation systems.

2.4.2 Efficiency Matrix Factorization without Sampling

The field of non-sampling matrix factorization has seen the development of various techniques aimed at overcoming its inefficiency. Relevant research includes the works of Chen et al. (2019), He et al. (2016), Xin et al. (2018), and Yuan et al. (2018). Among these, Chen et al. (2020) introduced a new loss objective designed to bypass computation across all user-item pairs. Their paper's Theorem 2.1 provides proof that for a generalized MF model equipped with a prediction function:

$$\widehat{y_{uv}} = h^T (p_u \circ q_v) \tag{3}$$

Within this framework, The respective latent vector representations for user u and item v, both in $\mathbb{R}^d$, are represented as $\boldsymbol{p_u}$ and $\boldsymbol{q_v}$. The symbol $\circ$ signifies the element-wise (Hadamard) product, and $\boldsymbol{h}$ is used to transform the interaction into a score prediction. Thus, the gradient of the loss function from Eq.(2) can be written as:

$$\tilde{L}(\Theta) = \sum_{u \in U} \sum_{v \in V^+} \left((c_v^+ - c_v^-) \widehat{y_{uv}^2} - 2 c_v^+ \widehat{y_{uv}} \right) + \sum_{i=1}^{d} \sum_{j=1}^{d} h_i h_j \left(\sum_{u \in U} p_{u,i} p_{u,j} \right) \left(\sum_{v \in V} c_v^- q_{v,i} q_{v,j} \right) \tag{4}$$

By rewriting the loss in this way, the model avoids explicit enumeration of all negatives and drastically reduces computational cost—from $\mathcal{O}(|\mathcal{U}| \cdot |\mathcal{V}| \cdot d)$ to $\mathcal{O}((|\mathcal{U}| + |\mathcal{V}|) \cdot \mathrm{d}^2 + |\mathcal{R}| \cdot \mathrm{d})$, making it feasible to train on large-scale implicit feedback datasets.

2.5 Evaluation Methodologies

2.5.1 Leave-one-out Evaluation Protocol

As an evaluation method in recommender systems, Leave-one-out is popular and particularly well-suited for Top-K recommendation scenarios. It is employed to learn the performance of recommender systems in various studies such as Chen et al. (2020), He et al. (2016), Chen et al. (2019), He et al. (2018), He et al. (2017), Xin et al. (2019), Ning et al. (2011), Rendle et al. (2009). For each user, one interaction is held out as the test instance, while the remaining interactions are used to train the model. Rather than ranking all items, a computationally intensive procedure, the system employs a sampling-based evaluation. This involves randomly selecting a small set of negative items, inserting the target item, and subsequently ranking these items based on their predicted scores.

This approach simulates real-world scenarios where the system must identify the most relevant item among many. HR and NDCG are evaluation metrics used to measure how well the model ranks the test item in its Top-K recommendations.

2.5.2 Hit Ratio (HR)

As a recall-based metric frequently employed to assess recommender systems, Hit Ratio (HR) has been used throughout in Chen et al. (2020), Chen et al. (2019), He et al. (2018), He et al. (2017), Xin et al. (2019), Ning et al. (2011), He et al. (2015), Deshpande et al. (2004). For each

user, a Top-K products list is created to check which products in the training batch appear in this list. The formula to calculate HR:

$$\mathrm{HR@K} = \frac{\text{Number of Hits @K}}{|\mathrm{U}|} \tag{5}$$

Where:

- $\mathrm{HR@K}$: Hit Ratio at rank K
- $|U|$: Total number of users evaluated.
- Hits @K: This metric counts the number of users for whom the relevant test item appears within the top K positions of their recommendation list.

A higher HR score reflects greater success in identifying and suggesting suitable items to users. The Hit Ratio (HR) value ranges from 0.0 to 1.0. An HR of 0.0 signifies that, for all users evaluated, their specific relevant item was not found within their top-K recommendation list. Conversely, an HR of 1.0 indicates perfect performance on this metric for the test set, meaning every user had their relevant item included in their top-K list.

2.5.3 Normalized Discounted Cumulative Gain (NDCG)

In recommender systems, it's not only important to know whether the correct item is recommended, but also where it appears in the recommendation list. Unlike Hit Ratio (HR), which only checks for the presence of the target item in the Top-K results, Normalized Discounted Cumulative Gain (NDCG@K) also accounts for the item's specific position in the ranking. Frequently employed across many studies such as Chen et al. (2020), Bayer et al. (2017), He et al. (2016), He et al. (2018), He et al. (2017), Xin et al. (2019) and He et al. (2015). This metric is considered crucial for real-world scenarios because it captures the trend of users to prioritize items presented earlier in a list. A correct recommendation at the top of the list is clearly more valuable than one buried at the bottom.

NDCG@K is a position-aware and precision-based evaluation metric. The scoring mechanism prioritizes relevant items found at higher positions in the ranked list, applying a discount that reduces the value contributed by relevant items at lower ranks. In practice, this encourages models to not only retrieve relevant items but also rank them properly. The metric is normalized to ensure that a perfect ranking would achieve a score of 1.0, while lower scores indicate suboptimal ranking performance, according to Järvelin & Kekäläinen (2002). Therefore, the value of NDCG ranges from 0 to 1, and the higher the NDCG score, the better the recommendation quality. During evaluation, NDCG@K is computed individually for each user, and the final performance is reported as the average across all test users. This makes NDCG a reliable and comprehensive measure of both ranking correctness and relevance prioritization.

2.6 Efficient Non-Sampling Factorization Machines (ENSFM)

The Efficient Non-Sampling Factorization Machine (ENSFM) is designed to optimize context-aware recommendation without relying on negative sampling. Instead of randomly selecting unobserved interactions, ENSFM processes the entire data space, which leads to more accurate and stable ranking results.

2.6.1 Overview

By eliminating negative sampling in the training of FM models, ENSFM aims to deliver quicker and more reliable learning, effectively optimizing the models for ranking performance. Based on the generalized embedding-based FM formulation presented in the studies by Xiao et al. (2017) and He et al. (2017), ENSFM employs a prediction function:

$$\widehat{y_{FM}}(x) = w_0 + \sum_{i=1}^{m+n} w_i x_i + h^T \sum_{i=1}^{m+n} \sum_{j=i+1}^{m+n} \left(x_i e_i \circ x_j e_j\right) \tag{6}$$

where:

- w_0 represents the global bias term,
- w_i is the feature i's weight,
- e_i is feature i's embedding vector,
- $\mathbf{h}$ is the neuron's weight in the predictive output layer.

The framework of ENSFM is shown in the Figure 1.

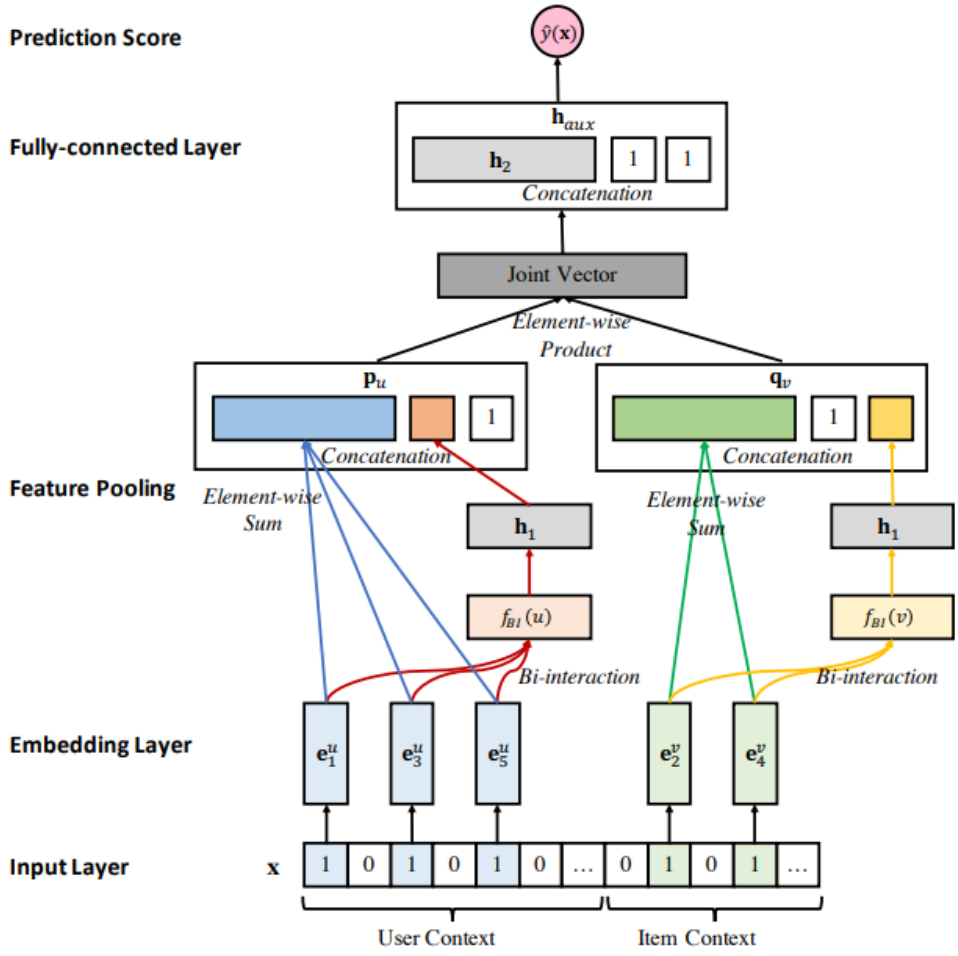


Figure 1. ENSFM Framework Architecture (Chen et al. 2020)

ENSFM transforms this FM formulation into a matrix factorization view with three key components:

i. Embedding Layer: Contextual features for users and items are embedded into latent vectors e^u, e^v, generating dense representations from sparse input.

ii. Memorization Strategy: The second-order term $f(x)$ is decomposed and reorganized to construct two auxiliary vectors p_u and q_v, representing user and item contexts respectively. The FM score is then rewritten as a matrix factorization-like function:

$$\widehat{y_{FM}}(x) = h_{aux}^{T}(p_u \circ q_v) \tag{7}$$

in which the auxiliary vectors p_u, q_v and h_{aux} represent user u, item v and prediction parameter, respectively.

This formulation avoids approximation while enabling efficient computation.

iii. Mini-batch Learning: A non-sampling training algorithm is proposed to update model parameters by considering all interactions, which improves both stability and convergence.

2.6.2 ENSFM Learning Algorithm

To train the ENSFM model efficiently without relying on negative sampling, the authors designed a mini-batch learning strategy that can update model parameters using all available data while keeping the computation cost under control. This strategy not only improves model performance but also ensures scalability for large-scale recommendation systems.

Instead of updating model parameters over the full user-item interaction matrix (which is computationally expensive), ENSFM works with small batches of users in each training step. For every batch, the algorithm generates auxiliary vectors that represent the user context and item context based on shared embeddings. The interaction signals between users and items are then captured by merging these vectors. The prediction score for each user-item pair is computed through this combination, which is formally described in Eq.(7) of this report.

After that, the model calculates a non-sampling loss function. This loss function uses confidence weights for each user-item interaction and includes both observed and unobserved entries. By doing so, it avoids the need for sampling negative examples and instead makes use of the entire data space. This leads to better optimization and more stable training compared to traditional sampling-based methods.

Algorithm 1 ENSFM Learning Algorithm

Require: Training data $\{\mathbf{Y}, \mathbf{U}, \mathbf{V}, \mathbf{X}\}$; weights of entries c; learning rate η; embedding size d

Ensure: Neural parameters Θ

Randomly initialize neural parameters Θ
while Stopping criteria is not met **do**
    while An epoch is not end **do**
        Randomly draw a training batch $\{\mathbf{Y_B}, \mathbf{U}, \mathbf{V}, \mathbf{X}\}$
        Build auxiliary vectors $\mathbf{P_B}$ for users
        Build auxiliary vectors $\mathbf{Q}$ for items
        Build auxiliary vector $\mathbf{h}$
        Compute the loss $\tilde{L}(\Theta)$
        Update model parameters
    end while
end while
return Θ

Figure 2. ENSFM Learning Algorithm (Chen et al. (2020)), which describes how the model is trained using a non-sampling strategy combined with mini-batch updates.

The entire process is illustrated in Algorithm 1 (see Figure 2). In the beginning, all neural parameters are randomly initialized. During each epoch, a batch of users is selected, and the corresponding auxiliary vectors for users and items are constructed. The model then calculates the prediction scores and evaluates the loss function before updating the parameters using gradient descent.

This training approach allows ENSFM to learn from the complete interaction set without losing important information due to sampling. It also maintains fast computation time by reusing shared computations and organizing the model into modular components.

3 IMPLEMENTATION

3.1 Technology

3.1.1 Python

Python is an interpreted, object-oriented programming language known for its ease of use and performance. It includes powerful built-in data structures and features dynamic typing and binding, enabling developers to rapidly prototype and develop applications. Python also effectively serves as a scripting or integration language, allowing different software components to be seamlessly combined. Its straightforward syntax and broad capabilities have made it widely popular in various domains such as data analytics, machine learning, and artificial intelligence.

One of Python's core advantages is its comprehensive standard library, which offers a wide array of tools for tasks such as data visualization (e.g., Numpy, Matplotlib), file handling, and networking. These built-in libraries are especially beneficial in areas like data science and machine learning, where Python has become a go-to language for both research and production.

This thesis employs the model proposed by Chen et al. (2020), which was originally implemented in Python 2.7. We have adapted the model and data loading functions for Python 3.9, as the latest Python version (3.13) is still undergoing bug fixes and isn't stable enough for use in this project. All coding tasks in this thesis were completed using Python.

3.1.2 Jupyter Notebook

Jupyter Notebook is a web-based platform designed for creating and sharing computational documents. It provides a streamlined, document-centric environment that integrates code, output, visualizations, and

markdown in a single interface. This makes it a widely-used tool in fields like Data Science, Computer Science, and Data Analysis.

When Anaconda is installed, Jupyter Notebook includes a default Python kernel. Additionally, each Conda environment operates as an independent environment within Jupyter Notebook, allowing you to use a separate kernel for each project. This setup is especially useful when working with different libraries for various projects, avoiding potential conflicts between them.

3.1.3 NumPy

NumPy is the core library of Python. It is used for manipulating numeric and matrix data. It provides a multidimensional array object — `ndarray` — and various efficient mathematical functions for fast operations on massive data. Thanks to being optimized with C, using NumPy is much faster than a pure Python list.

NumPy supports vectorized operations, which help shorten the code and increase performance. It is the foundation of many other libraries such as Pandas, Scikit-learn, and TensorFlow.

In this thesis project, we use NumPy for listing the information that we want to know about the datasets in the preprocessing datasets step.

3.1.4 Pandas

Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool, built on top of the Python programming language, according to Team, P. D. (n.d.). Pandas highlights include providing two main data structures: Series (one-dimensional `ndarray` with axis labels) and DataFrame (a table including columns), very similar to Excel spreadsheets or tables in a database.

Pandas makes data manipulation easy and intuitive, including: reading/writing data from CSV files, Excel, SQL; filtering, sorting, groupby,

handling missing data, quick statistics, and converting data types. Based on those functions, Pandas becomes an indispensable tool in data science projects, business analytics, and data preprocessing for machine learning.

This library is built on NumPy, so it is fast and compatible with other libraries like Matplotlib, Scikit-learn, and TensorFlow. In this thesis project, we use Pandas for loading JSONL data into a DataFrame, inspecting the structure and content of the data, and counting unique values in specific columns.

3.1.5 Matplotlib

Matplotlib is a widely-used Python library designed for data visualization, capable of producing various types of graphs. It supports creating static, dynamic, and interactive plots, including line charts, bar charts, pie charts, scatter plots, heatmaps, and numerous other sophisticated visualizations.

This library works well with NumPy and Pandas so it is very suitable for analyzing data and displaying output. With Matplotlib, you can easily edit the titles, axis labels, colors and save your graphs as images (PNG, PDF,…).

Matplotlib is a foundation of advanced visualization libraries like Seaborn, Plotly. Despite having more complex syntaxes than other libraries, it is very flexible and powerful. In this thesis, we use this technique for visualizing data to have a clear, informative overall (the trend, spread,…) of the datasets and removing outliers of the datasets – top 1% of values.

3.1.6 TensorFlow

According to Developers, T. (2025, March 17), TensorFlow is a flexible, open-source machine learning framework that offers extensive tools, libraries, and community support, enabling researchers and developers

to advance machine learning techniques and implement ML-driven applications. Initially developed by Google's Machine Intelligence team for neural network research, TensorFlow has since become applicable across diverse fields Developers, T. (2025, March 17).

Chen et al. (2020) utilized TensorFlow version 1.x for their original recommender model. In this thesis, the model was updated to use a more recent version of TensorFlow to improve compatibility and functionality with new datasets.

3.1.7 Google Colab and Kaggle

Google Colab is a free online coding platform that allows you to use Google hardware, especially GPUs and TPUs, to train machine learning models right in your browser. Instead of having a powerful personal computer, you can take advantage of Colab’s computing power to run deep learning models without complex settings.

Kaggle also provides a free online coding environment with GPU that helps you train machine learning models directly on their platform. Users can write code using Jupyter Notebook, access diverse datasets, and take advantage of Kaggle’s server resources to save time and effort compared to using a personal computer.

In this thesis, I took advantage of the free 30-hour Kaggle GPU P100 and bought Google Colab Pro to use the A100 GPU for training the recommender model.

3.2 Datasets

For the performance evaluation of the ENSFM recommender system, we utilized two large-scale, real-world datasets specifically chosen for their

abundance of features: Yelp2018[2] and Amazon Books Reviews 2023[3]. We put all of the statistics of all datasets, including the datasets of the study Chen et al. (2020) in the Appendix 1. Statistical details of the datasets.

We selected the three categories with the most items from the initial top five list. This decision was driven by limitations in computational capacity, which wasn't sufficient for the dataset's size, and by the considerable time required for training and testing. Then, we combined them as a multi-category dataset that represents the entire original data. In addition, we also separated each category from the multi-category dataset into separate datasets. This helps us easily compare and evaluate the effectiveness of the model between using a single category and multiple categories.

We will briefly introduce the original datasets first and describe the datasets used in this thesis for evaluating the ENSFM model working with big, real-world datasets.

3.2.1 Original Datasets Information

As you can see in Table 2, the statistics of the Yelp2018 and Amazon Books Reviews 2023 datasets are very large. To train the model with these datasets, we need powerful computing resources, which we do not have, so we decided to split them into a smaller dataset — a multi-category dataset that still represents the original.

Table 2. Source Datasets Statistics

Dataset	#User	#Item	#Features	#Interaction	#Field

[2] https://business.yelp.com/data/resources/open-dataset/
[3] https://amazon-reviews-2023.github.io/

Yelp2018	1,987,896	150,346	5,976,321	6,990,280	9
Amazon Books Reviews 2023	10,3M	4,4M	145,611,112	29,5M	10

a) Yelp2018 Dataset

According to Inc., Y. (n.d.), the Yelp Open Dataset is a subset of Yelp data that is intended for educational use. It provides real-world data related to businesses including reviews, photos, check-ins, and attributes like hours, parking availability, and ambience. In this thesis, we use the Yelp2018 dataset that does not contain the photos folder.

We chose this Yelp2018 dataset because of many reasons. First of all, this dataset has user contexts (name, history compliments, average stars that user rated,…). Next is because it has user-items interactions. In this case it is reviews. Moreover, it also has the item - business contexts (name, address, city, longitude, latitude,…). And of course, this is a real-world e-commerce dataset which is very vital factor to this choice.

This dataset includes 1 PDF file about the documentation and terms and five JSON files (8.65 GB of data). By using programming, we can figure out the keys in each JSON file easily. Because of the length of the code, it is made available in the appendices section – Appendix 2. Extract Unique JSON Keys from Yelp2018 Dataset.

- `business.json`: This JSON file contains business information. You can see its unique keys in the file in Figure 3.

```
🔑 Keys in business JSON:
- address
- attributes
- business_id
- categories
- city
- hours
- is_open
- latitude
- longitude
- name
- postal_code
- review_count
- stars
- state
```

Figure 3. Unique keys of Yelp2018 dataset - `business.json` file

- `review.json`: Inside this JSON file, you'll find the complete text for each review, alongside the specific `user_id` of the person who wrote it and the `business_id` for the business being reviewed.You can see its unique keys in the file in Figure 4.

```
🔑 Keys in review JSON:
- business_id
- cool
- date
- funny
- review_id
- stars
- text
- useful
- user_id
```

Figure 4. Unique keys of Yelp2018 dataset - `review.json` file

- `user.json`: User data comprises the user's friends list and related user metadata. You can see its unique keys in the file in Figure 5.

```
🔑 Keys in user JSON:
- average_stars
- compliment_cool
- compliment_cute
- compliment_funny
- compliment_hot
- compliment_list
- compliment_more
- compliment_note
- compliment_photos
- compliment_plain
- compliment_profile
- compliment_writer
- cool
- elite
- fans
- friends
- funny
- name
- review_count
- useful
- user_id
- yelping_since
```

Figure 5. Unique keys of Yelp2018 dataset - `user.json` file

- `checkin.json`: This JSON file contains checkins of businesses. You can see its unique keys in the file in Figure 6.

```
🔑 Keys in checkin JSON:
- business_id
- date
```

Figure 6. Unique keys of Yelp2018 dataset - `checkin.json` file

- `tip.json`: This JSON file contains tisp, which constitute advice offered by users regarding businesses. Unlike reviews, these tips

are usually shorter and designed to give fast suggestions. Figure 7 shows the unique keys in the file.

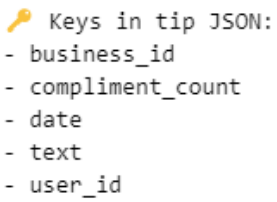


Figure 7. Unique keys of Yelp2018 dataset - `tip.json` file

b) Amazon Books Reviews 2023 Dataset

Amazon Books Reviews 2023 dataset is a category named Books in a bigger dataset – Amazon Review'23 which was published in the study Hou et al. (2024). Containing book reviews posted on Amazon between May 1996 and September 2023, the Amazon Books Reviews 2023 dataset constitutes a large-scale resource for e-commerce analysis.

The reasons I chose this dataset are the same as those for the Yelp2018 dataset. In this Amazon Books Reviews 2023 dataset, it has user contexts, item - book contexts, huge amounts of user-item interactions and it is a real-world e-commerce dataset.

This dataset includes 2 JSONL files – `Books.jsonl` (5.8GB) and `meta_Books.jsonl` (4.6GB). With the experiences working with Yelp2018 dataset, we can easily extract the keys of each JSONL file. Because of the length of the code, they are put in the appendices section – Appendix 3. Extract Unique JSONL Keys from Amazon Books Review 2023 Dataset. Because of the file size of Books.jsonl file, we loaded first 1000 data rows to have a briefly view about the keys of the file. But with the meta_Books.jsonl file, we loaded entire data.

- `Books.jsonl`: This JSONL file contains user interactions data. In this case, each line represents a book review of a user. You can see its unique keys in the file in Figure 8.

```
✅ Columns in Books.jsonl:
Index(['rating', 'title', 'text', 'images', 'asin', 'parent_asin', 'user_id',
       'timestamp', 'helpful_vote', 'verified_purchase'],
      dtype='object')
```

Figure 8. Unique keys of Amazon Books Reviews 2023 dataset - `Books.jsonl` file

- `meta_Books.jsonl`: This JSONL file contains detailed metadata for books. According to Lab, M. (n.d.), some items lack metadata. You can see its unique keys in the file in Figure 9.

```
Columns in meta_Books.jsonl:
Index(['main_category', 'title', 'subtitle', 'author', 'average_rating',
       'rating_number', 'features', 'description', 'price', 'images', 'videos',
       'store', 'categories', 'details', 'parent_asin', 'bought_together'],
      dtype='object')
```

Figure 9. Unique keys of Amazon Books Reviews 2023 dataset – `meta_Books.jsonl` file

3.2.2 Single and Multi-category Datasets

For each original dataset, we separated it into one multi-category dataset and three single-category datasets. Moreover, we did not use all features/files but extracted only the necessary information to train the model. The methods and steps we used to preprocess these datasets will be described more clearly in part 3.4.4. Data Preprocessing.

a) Single and Multi-category Datasets of Yelp2018 Dataset

In the Table 3, you can see the statistical details of all datasets that relate to Yelp2018 original dataset.

Table 3. Statistical details of the single and multiple categories dataset that relate to Yelp2018 dataset

Dataset	#User	#Item	#Features	#Interaction	#Field

Multi-category	21,114	50,673	75,495	277,417	11
Shopping	2,908	11,164	16,809	32,655	10
Food	10,684	18,204	32,524	121,963	10
Restaurant	32,590	31,722	68,339	381,429	10

- Multi-category dataset:

By programming, we can make a list of Most Common Business Categories in `business.json`. By using this list, we chose top 1st to top 3rd of that list and combine them as Yelp2018 multiple categories dataset.

```python
import json
from collections import Counter

category_counter = Counter()

with open('yelp_business.json', 'r', encoding='utf-8') as f:
    for line in f:
        data = json.loads(line)
        categories = data.get('categories')
        if categories:
            for cat in categories.split(','):
                category_counter[cat.strip()] += 1

# Sort by count
sorted_categories = category_counter.most_common()

# Print out
for category, count in sorted_categories:
    print(f"{category}: {count}")
```

```
Restaurants: 52268
Food: 27781
Shopping: 24395
Home Services: 14356
Beauty & Spas: 14292
```

Figure 10. Most Common Business Categories list of Yelp2018 dataset

To know more about the statistics of this dataset, you can see more in the Table 3.

- Single-category dataset:

We evaluated the difference in model performance between multi-category and single-category learning by treating each category within the multi-category dataset as an individual dataset. In this case, they are the Restaurants dataset, Food dataset, Shopping dataset.

b) Single and Multi-category Datasets of Amazon Books Reviews 2023 Dataset

Table 4. Statistical details of the single and multiple categories dataset that related to Amazon Books Reviews 2023 dataset

Dataset	#User	#Item	#Features	#Interaction	#Field
Multi-category	57,350	62,503	119,882	298,738	10
Arts & Photography	10,180	10,690	20,690	33,781	9
Genre Fiction	35,391	38,709	74,126	211,966	9
Children's Books	30,183	36,299	67,928	160,309	9

As with the Yelp dataset, in this dataset, we listed the Book Category Distribution and chose the 2nd to 4th top categories as the multiple categories dataset. We didn't choose the 1st because the number of books in those categories is too large for our computing power. We also treated each category as a separate dataset and compared the learning results of single-category datasets to the multiple categories ones. You can see the code for listing the Book Category Distribution in Appendix 4. Listing the Book Category Distribution in the Amazon Books Reviews 2023 dataset.

3.3 Testing methodology

In this thesis, we use the same evaluation method – leave-one-out – as in the study by Chen et al. (2020). This method is widely used to test the performance of item recommendation, according to Chen et al. (2020). Specifically, for each user, the system will keep the most recent

interaction as the test sample, while other interactions are used for training the model.

To evaluate the ranking list quality, two popular metrics that were used in the study by Chen et al. (2020) are HR and NDCG, which I mentioned in 2.5.2 and 2.5.3 respectively. As in the original study – Chen et al. (2020), they used those two metrics with the lengths of K = 5, 10, 20.

Using HR and NDCG allows us to evaluate the system more comprehensively. Not only can we test the ability to recommend suitable items, but we can also evaluate the rank position of the correct item. These are the evaluation standards that were verified in many studies and are suitable for the purpose of evaluating modern recommendation systems.

3.4 Procedures

3.4.1 System Version Upgrade

As described in 3.1.1, Chen et al. (2020) used Python 2.7 for their main model and other files of code. When using their codes, we need cho convert Python 2.7 code to the new version – here, we chose version 3.9, not the latest because the latest version is not stable to use and may have many bugs.

As same as Python, the version of TensorFlow is outdated and we had to update it to a newer version.

3.4.2 Re-experimentation

After updating the code version, we implemented again the training and testing steps to check the performance of the model. The regeneration results were immediately compared to the data presented by the authors to guarantee consistency. The HR and NDCG values were the same as the original model and the training time was different because of the difference in computing power.

In addition, not only we use the personal computer to train the model, but also we use Kaggle and Colab Pro to train so that we also regererated the model with the original datasets in the study Chen et al. (2020) by using Kaggle and Colab Pro environment. The results were consistent across all these environments, confirming that the code updates and execution environment improvements did not change the original logic.

3.4.3 Dataset Selection

To shorten the searching time and improve the efficiency of searching, we reviewed all the articles that cited the original article by Chen et al. (2020). Because of having the same topic, the possibility of reusing the datasets used in those articles was very high. After this process, we found the two most suitable datasets — Yelp2018 and Amazon Book Reviews 2023. To confirm that these were the datasets we were looking for, we read the dataset documentation and examined the data structure of each dataset.

After all, these datasets met the necessary criteria and were also suitable for the data structure and requirements of the ENSFM model.

3.4.4 Data Preprocessing

In the original article, the authors did not provide the preprocessing steps so we had to read papers relevant to this topic or cite this paper but we had no clues about the preprocessing steps of the original datasets – Frappe, Last.fm and MovieLens. Because of that, we built a preprocessing structure in case of applying new datasets. After preprocessing two datasets, we shortened the process to the most basic, general steps for processing the two datasets to apply to this RS model (Figure 12). Not all steps and requirements are the same for all datasets, but the sequence of preprocessing new datasets in this thesis will be the same. You can find all the preprocessing code of these datasets in Github.

The data preparation process starts with setting up the environment and configuring the processing pipeline through command-line arguments. These arguments specify crucial details such as the locations of input files (containing item information and user interactions), thresholds for minimum interactions per item and user, the desired processing mode ('separate' for single categories or 'unified' for multiple), and options for subsampling or balancing data. Target categories for analysis are defined here. Following this setup, the item data processing begins, often involving an initial pass to calculate outlier thresholds based on item features like price or rating counts. For 'unified' mode, these thresholds are calculated individually for each target category, while in 'separate' mode, a single threshold is determined for the specific category being processed.

With thresholds established, the main item processing pass filters the items. Items are retained only if they belong to the target category (or categories in unified mode), meet the minimum interaction threshold, and do not exceed the calculated feature outlier limits (using category-specific limits in unified mode). A mapping from category names to integer IDs is generated, particularly for unified processing. Valid items are assigned unique sequential integer IDs, and relevant features (like average ratings or price) are extracted from the metadata. A key distinction for 'unified' mode is the addition of the item's primary category ID as an extra feature. All extracted features, including the assigned item ID and the potential category ID, undergo numerical transformations using scaling and offsets to ensure they occupy distinct value ranges before being stored.

Next, the user interaction data is loaded and processed. This step might involve reading from one or multiple files depending on the dataset structure. Interactions are filtered rigorously: only interactions involving items that passed the previous filtering step are kept, and interactions from users who do not meet a minimum total interaction count are dis-

carded. Users associated with the remaining valid interactions are assigned unique sequential integer IDs. Relevant user features (such as average ratings, total interactions, or helpfulness scores) are extracted either from dedicated user files or the interaction records themselves. Similar to item features, these user features are numerically transformed using scaling and offsets. The valid interactions for each user, typically including timestamps and item identifiers, are collected and stored, often grouped by user ID. For 'unified' mode processing, interaction counts per category are also tracked during this stage.

Once the initial user and interaction data is loaded and filtered, the interaction lists undergo refinement. First, each user's sequence of interactions is sorted chronologically based on timestamps, which is essential for time-based train/test splitting later. After sorting, two optional steps might be applied. If processing in 'unified' mode and the category balancing option is enabled, the interactions across different categories are balanced, usually by probabilistically up-sampling interactions from less frequent categories to mitigate data imbalance. Subsequently, if a subsampling ratio is specified, stratified subsampling is performed, where a fraction of users is randomly selected from groups based on their interaction frequency, reducing the overall dataset size while preserving user activity distributions.

The final steps involve re-indexing (primarily for unified mode), splitting the data, and saving the formatted output. A crucial step for 'unified' mode is global re-indexing: new, dense, sequential integer IDs (starting from 1) are created for *only* those users and items that remain after all filtering, balancing, and subsampling stages. The stored feature lists are updated with these new dense IDs. The structure of feature input will be like this: `"userID-feature1-feature2-..."`,`"itemID-feature1-feature2-..."`.

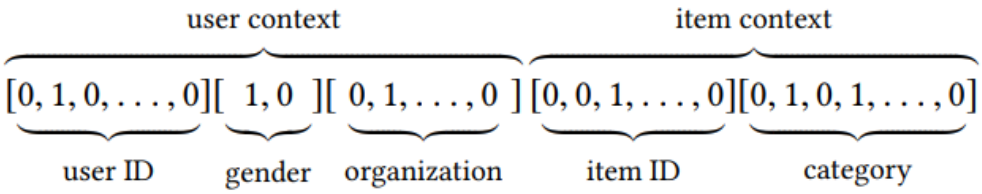


Figure 11. An example of feature input for each user and item (Chen et al. 2020, p.2)

Then, the interaction data is split into training and testing sets for each user (who must have at least two interactions remaining). Typically, the last interaction chronologically becomes the test instance, while all preceding interactions form the training set, although random selection for the test instance is also an option. Finally, the processed data is formatted and saved. For each interaction in the train and test sets, the corresponding user and item feature lists (containing the final transformed and potentially re-indexed IDs) are retrieved, converted into a specific string format (e.g., dash-separated integers), and written as rows to `train.csv` and `test.csv` files within the designated output directories (either category-specific or a single unified directory).

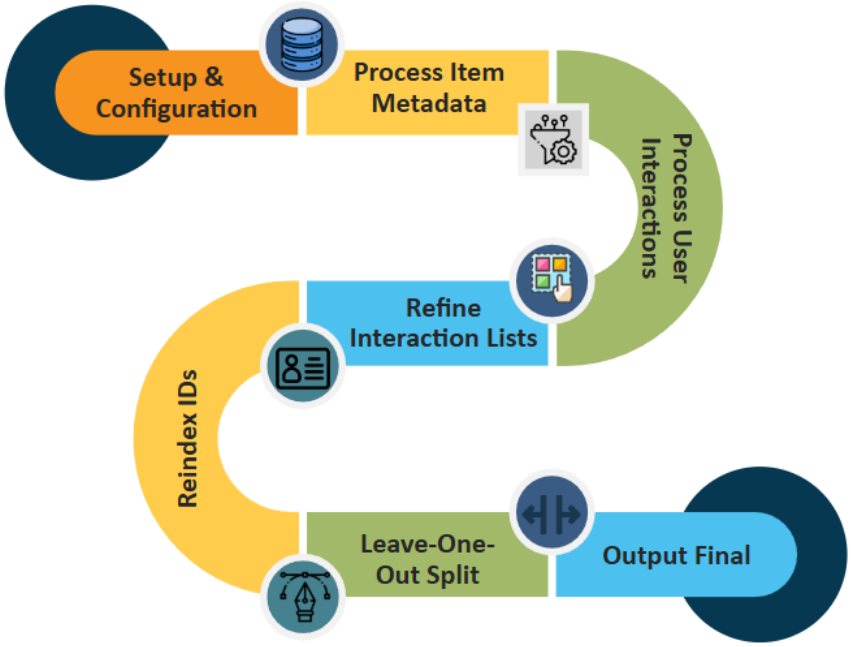


Figure 12. General preprocessing steps for this thesis datasets

a) Preprocessing Yelp2018 dataset

For both types of datasets – single category datasets and multi-categories dataset, we combined data from three JSON files (`business.json`, `user.json` and `review.json`) together to create them. First, the process begins by setting up the necessary parameters through command-line arguments. This includes defining input file paths for the raw Yelp data (users, businesses, reviews), specifying minimum review thresholds, choosing whether to sub-sample the data, selecting the processing mode ('`separate`' for individual categories or '`unified`' for multiple categories), designating an output directory, defining the method for splitting data into test sets, and deciding whether to balance categories in unified mode. The target categories for single-category processing (Food, Restaurants, Shopping) are also defined here. Initially, the system loads user data from the `user.json` file. Each user is assigned a unique numerical ID, and a mapping from the original ID string is kept. Key features like average stars, review count, 'useful' votes, and years marked as 'elite' are extracted and transformed into distinct numerical ranges for later use as model features.

Next, the business data is loaded from `business.json`, which involves two passes. In the first pass, the script identifies businesses belonging to the specified target categories (Food, Restaurants, Shopping for separate mode, or all three for unified mode). It calculates thresholds for star ratings and review counts within each category (based on the 75th percentile) to identify and later remove outliers. In the second pass, businesses are filtered based on their primary category (the first alphabetically if multiple apply) and whether they fall below the calculated outlier thresholds. Kept businesses are assigned unique numerical IDs (offset from user IDs), and features like stars, review count, city (which also gets a unique ID), and open status are extracted and numerically transformed. Crucially, for the unified dataset processing, an additional feature representing the business's primary category (Food, Restaurants, or Shopping) is added to its feature list.

After processing users and businesses, the review data is loaded from `review.json`. Each review's timestamp is extracted, and the review is kept only if both the associated user and business IDs exist in the sets created in the previous steps (meaning the user is valid and the business belongs to a target category and is not an outlier). These valid interactions (reviews) are stored, grouped by user ID. Following this initial loading, the interactions undergo a frequency filtering step. Users who have fewer than a specified minimum number of reviews (`min_user_reviews`) after the initial filtering are removed, along with their associated interactions, ensuring that only sufficiently active users remain in the dataset.

The next steps involve optional balancing and subsampling, followed by re-indexing. If processing in 'unified' mode and the balancing option is enabled, the interactions are balanced across the different categories (Food, Restaurants, Shopping). This is done by calculating sampling rates to up-sample interactions from less frequent categories, aiming for a more even distribution. After potential balancing, if a `subsample_ratio` less than 1.0 was provided, stratified subsampling is performed. Users are grouped by their interaction count, and a random fraction of users is selected from each group to create a smaller, representative dataset. Finally, both users and businesses that remain after all filtering, balancing, and subsampling are reindexed. This creates new, dense, sequential IDs (starting from 1 for users, and an offset for businesses) only for the active entities in the final interaction set, which is important for efficient model training.

Finally, the process involves splitting the processed data into training and testing sets and saving the output. For each user remaining in the dataset (with at least two interactions), their interactions are sorted by time. By default, the very last interaction is held out for the test set, while all preceding interactions form the training set. Alternatively, a random interaction can be selected for the test set if this option is specified. These train and test interactions are then formatted for output. For

each interaction, the corresponding re-indexed feature lists for the user and the business are retrieved. These lists of numerical feature IDs are converted into dash-separated strings. Finally, these strings are written to `train.csv` and `test.csv` files. In 'separate' mode, these files are saved within directories named after each category (Food, Restaurants, Shopping). In 'unified' mode, they are saved in the specified unified output directory.

The structure of model input format in single and multi-category are quite similar to each other. Both are the combination of user features and items features but for items features in multi-category dataset have category feature. In single category dataset, the structure for each line in the output CSV is `'new_user_id - unique_useful - unique_avg_star - unique_review_count - elite_years', 'new_business_id - unique_avg_star_biz - unique_review_count_biz - city_id_biz - is_open'` and with multiple categories dataset, it is `'new_user_id - unique_useful - unique_avg_star - unique_review_count - elite_years', 'new_business_id - unique_avg_star_biz - unique_review_count_biz - city_id_biz - is_open - category_id_biz'`.

b) Preprocessing Amazon Books Reviews 2023 dataset

For both types of datasets – single category datasets and multi-categories dataset, we combined data from all JSONL files that have in the original Amazon Books Reviews 2023 (`meta_Books.jsonl` and `Books.jsonl`) together to create them. The preprocessing workflow starts with initialization and parsing command-line arguments. This step defines essential parameters like the paths to the input metadata and user interaction files, minimum review counts required for items and users to be included, and options for data sub-sampling. It also determines the processing mode – either `'separate'` to create individual datasets for specific book categories or `'unified'` to create one dataset

covering multiple categories. Based on the chosen mode, target category lists are defined, and the relevant processing function is called. For item (book) processing using the metadata file, the first task is calculating outlier thresholds. In `'separate'` mode, this involves streaming the metadata once per target category, filtering items by that category and minimum reviews, and then calculating the 75th percentile for `rating_number` and `price` across that single category. In `'unified'` mode, the metadata is streamed, and these percentile thresholds are calculated individually for each target category the items belong to.

Continuing with item processing, the metadata file is streamed again (or for the main pass in unified mode) to filter the books and create their feature representations. Items are kept if they belong to the target category (separate mode) or any of the target categories (unified mode), meet the minimum review count, and fall below the previously calculated outlier thresholds for `rating_number` and `price` (using category-specific thresholds in unified mode). Each valid item is assigned a unique sequential integer ID. Item features like average rating, total rating count, and price are extracted. Importantly, specific offsets and scaling are applied to these features (and the item ID itself) to place them into distinct numerical ranges, preparing them for model input. For the `'unified'` dataset only, an additional integer ID representing the item's primary category (e.g., the first alphabetically if it belongs to multiple) is generated, scaled with its offset, and added to the item's feature list.

Next, user interactions are processed using the `Books.jsonl` file. This file is streamed, and interactions are filtered based on two main criteria: the interaction's item (book) must be present in the set of valid items created in the previous step, and the user must meet a minimum total review count threshold (`min_user_reviews`). Users passing these filters are assigned unique sequential integer IDs upon their first appearance. At this point, user features like total helpful votes, average rating across their reviews, total review count, and the ratio of verified purchases are extracted. Similar to item features, these user features are scaled and

offset into distinct numerical ranges. The interaction details (specifically, the item ASIN and the timestamp) are stored, grouped by the user ID. In `'unified'` mode, the system also keeps track of interaction counts per category, which is used later for balancing.

After loading and initial filtering, the interaction lists for each user are refined. First, each user's interactions are sorted based on their timestamps. Then, two conditional steps may occur. If processing in `'unified'` mode and the balancing option is active, category balancing is performed. This involves calculating sampling rates to probabilistically duplicate interactions belonging to less frequent book categories, aiming for a more even distribution across categories in the final dataset. Following potential balancing, if a sub-sampling ratio less than 1.0 was specified, `stratified sub-sampling` is applied. Users are grouped based on their number of interactions, and only a random fraction of users from each group is kept, reducing the dataset size while trying to maintain representation.

Finally, the data undergoes reindexing (specifically in unified mode), is split into training and testing sets, and saved. In `'unified'` mode, a global reindexing step occurs: all unique user IDs and item ASINs remaining after balancing and subsampling are assigned new, dense, sequential integer IDs (starting from 1). The feature dictionaries are updated with these new IDs. (In `'separate'` mode, reindexing happens implicitly within each category's independent processing). To create the train/test split, each user's chronologically sorted interactions are considered. By default, the last interaction is designated for the test set, while all others go into the training set (an option exists to use a random interaction for testing instead). For saving, the system retrieves the final feature lists (using re-indexed IDs in unified mode) for the user and item in each interaction. These feature lists are formatted into dash-separated strings and written to `train.csv` and `test.csv` files, placed either in category-specific subdirectories (separate mode) or the designated output directory (unified mode).

The structure of input format in single and multi-category are quite similar to each other. Both are the combination of user features and items features but for items features in multi-category dataset have category feature. In single category dataset, the structure for each line in the output CSV is `'new_user_id - helpful_vote_feat - user_avg_rating_feat - user_review_count_feat - user_verified_ratio_feat', 'new_item_id - item_avg_rating_feat - item_rating_count_feat - item_price_feat'` and with multiple categories dataset, it is `'new_user_id - helpful_vote_feat - user_avg_rating_feat - user_review_count_feat - user_verified_ratio_feat', 'new_item_id - category_id_feat - item_avg_rating_feat - item_rating_count_feat - item_price_feat'`.

3.4.5 Model Training and Evaluation

After preprocessing, the train.csv and test.csv files containing user and item feature strings are loaded by the LoadData class. This class reads the CSVs, maps unique feature strings to simple integer IDs ("binding"), determines the size of the feature spaces, and prepares the data structures needed by the model.

The prepared data is then passed to the ENSFM model, which is built using TensorFlow. The ENSFM class defines the model's architecture, including learnable user and item embeddings and biases, used to calculate prediction scores based on the input features.

The model is trained over a set number of epochs using an Adagrad optimizer to minimize a loss function designed for implicit feedback data, which incorporates negative weighting. Training proceeds in batches, updating model parameters iteratively. During this process, the model's performance is periodically evaluated on the test dataset using the evaluate function. Evaluation measures recommendation quality using standard Top-K metrics: HR and NDCG at K=5, 10, and 20.

It's important to note that the provided LoadData and ENSFM model code, including their core logic and evaluation procedures, were used as-is from the original authors. The training of the ENSFM model on the preprocessed datasets utilizes specific hyperparameters to control the learning process. These parameters include:

- Batch size: 512
- Embedding size: 64
- Learning rate: 0.05
- Verbose: 100
- Dropout: 0.9
- Negative weight: 0.05
- Epochs: 501 (with evaluation every 100 epochs)
- Metric: Top-K evaluation using HR and NDCG at K values of 5, 10, and 20.

4 RESULTS

This section is dedicated to addressing the two research questions posed in this study, presenting the experienced findings derived from experiments conducted on the two new datasets. All training processes were implemented on Tesla P100-PCIE-16GB GPU. A big consideration here is that the preprocessing steps for the Yelp2018 datasets and Amazon Books Reviews 2023 dataset may be different from the datasets in the original article.

4.1 ENSFM Model Results on the New Dataset (Research Question 1)

As seen in Table 5, the training results of new datasets are not good when comparing to the original ones. Both HR and NDCG scores consistently increase as the recommendation list length K grows from 5 to 20 across all datasets. The evaluation results of multi-category datasets (for both Yelp2018 and Amazon Books Reviews 2023 dataset) are much lower than the single category datasets. From this, we can say that the performance of this model will be better when applied to a single category dataset like only recommending restaurants or hotels or movies for users.

Table 5. Evaluation Results

Dataset	HR@5	HR@10	HR@20	NDCG@5	NDCG@10	NDCG@20
Frappe	0.6094	0.7118	0.7889	0.4771	0.5105	0.5301
Last.fm	0.3683	0.4729	0.5793	0.2744	0.3082	0.3352

Mov-ielens	0.0601	0.1024	0.1690	0.0373	0.0508	0.0674
Multi Yelp2018	0.0294	0.0491	0.0796	0.0186	0.0250	0.0326
Food	0.05	0.0823	0.1323	0.0312	0.0416	0.0541
Restau-rants	0.0378	0.0628	0.1015	0.0236	0.0317	0.0414
Shop-ping	0.0512	0.0814	0.1218	0.0339	0.0436	0.0538
Multi Amazon	0.0240	0.0354	0.0508	0.0163	0.0199	0.0238
Genre Fiction	0.0412	0.0609	0.0849	0.0281	0.0345	0.0405
Chil-dren's Books	0.0448	0.0628	0.0846	0.0311	0.0369	0.0424
Arts & Photog-raphy	0.0549	0.0744	0.0982	0.0385	0.0447	0.0508

Because of space limitations, we only show the HR@10 metric results of all the datasets.

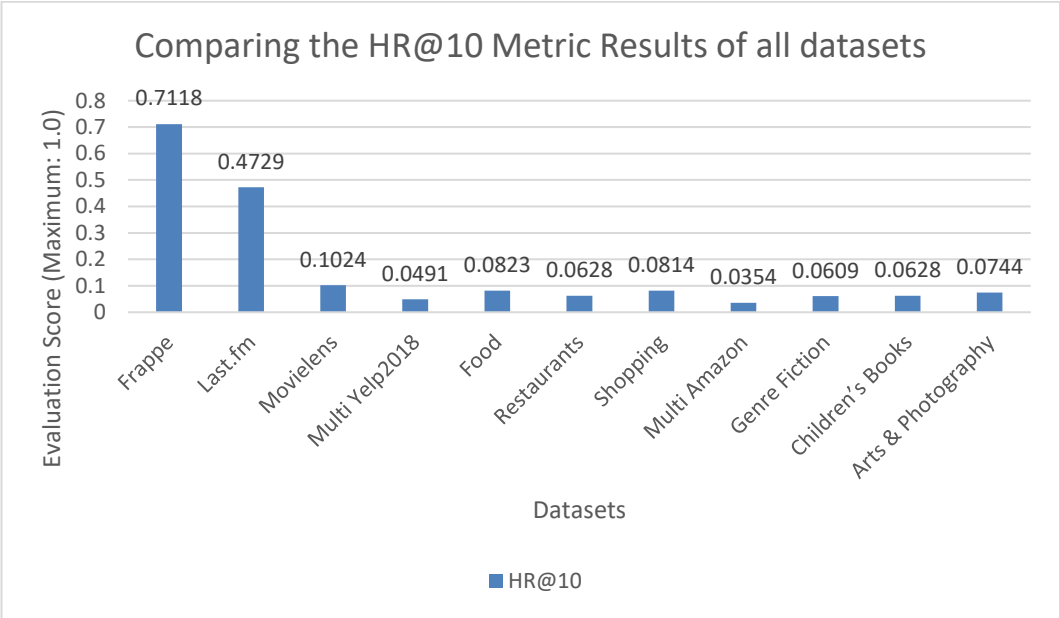


Figure 13. Performance of the ENSFM model on all the datasets

As well as the training time of new datasets are much longer than the datasets in the Efficient Non-Sampling Factorization Machines for Optimal article but the results are not better. It takes more than 4 hours and 3 hours to train Multiple Categories datasets of Yelp2018 and Amazon Books Reviews 2023 datasets, respectively but the performance of them are not good when taking much more training time. See the statistics in Figure 14 and Table 6 for more details.

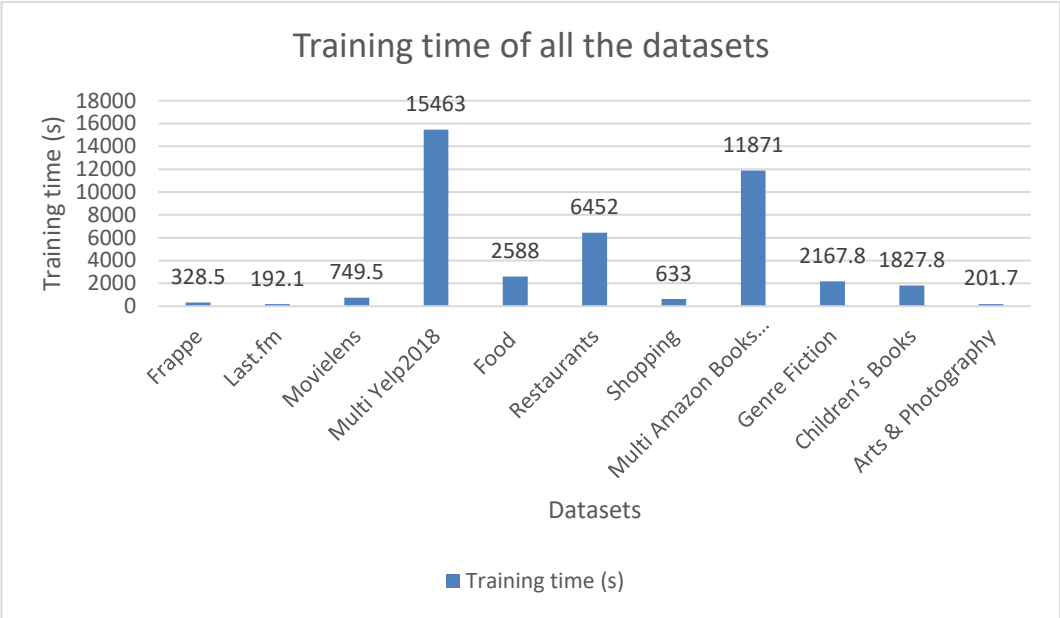


Figure 14. Comparison of training time of each dataset

For more information, the following table shows how many hours and minutes it takes to train datasets.

Table 6. Statistical details of the training time of every dataset on GPU Tesla P100-PCIE-16GB

Dataset	Running time
Frappe	5 minutes 28.5 seconds (328.5 seconds)
Last.fm	3 minutes 12.1 seconds (192.1s seconds)
Movielens	12 minutes 29 seconds (749.5 seconds)
Multi Yelp2018	4 hours 17 minutes 43 seconds (15463 seconds)
Food	43 minutes 8 seconds (2588 seconds)

Restaurants	1 hour 47 minutes 32 seconds (6452 seconds)
Shopping	10 minutes 33 seconds (633 seconds)
Multi Amazon Books Reviews 2023	3 hours 17 minutes 51 seconds (11871 seconds)
Genre Fiction	36 minutes 7.8 seconds (2167.8 seconds)
Children's Books	30 minutes 27.8 seconds (1827.8 seconds)
Arts & Photography	3 minutes 21.7 seconds (201.7 seconds)

4.2 Key Factors Affecting the ENSFM Model's Performance on the New Dataset (Research Question 2)

Based on the evaluation results, training time and statistics of the datasets, we can determine these factors that directly affect the performance of the model.

The most vital factor appears to be the high sparsity and large scale of the Yelp and Amazon datasets compared to the datasets in the original article (Frappe, Last.fm, Movielens). Even after preprocessing, these datasets have many more users and items, but the number of interactions per user/item is much lower. This sparsity makes it very difficult for the model to learn reliable user preferences and item relationships, leading directly to lower HR and NDCG scores.

The Yelp and Amazon datasets possess a substantially larger number of features (due to diverse item attributes, user details, and contextual information like city IDs). This increased complexity, especially when

combined with high sparsity, makes the learning task harder for the model and can hinder its ability to identify the most predictive feature interactions. It also contributes significantly to the longer training times.

After many experiments, we found that having much contextual information for users and items does not increase the performance of the model but choosing the features that are suitable for the purposes of the model will create the better performance.

The single-category datasets generally performed slightly better than their multi-category counterparts. This suggests that the increased focus within a single category leads to a less sparse, less complex problem with potentially more homogeneous user behavior, making it an easier task for the model compared to the combined and balanced multi-category datasets.

5 SUMMARY AND REFLECTION

This study looked at the ENSFM model to see how well it worked on big, sparse datasets from Yelp and Amazon Books. We prepared the data in different ways (single or multiple categories) to check if ENSFM works well in real situations, not just on standard test data. We looked at how accurate it was (using HR and NDCG) and how long it took to train. We found that the model was not as accurate and took much longer to train compared to earlier tests, showing problems caused by the size of data, sparsity (few interactions), and complex features.

Although this study gave us useful information, it's mainly a first look at using ENSFM with this difficult data. It sets a starting point for more research, not a ready-to-use solution.

Future research should try different ways to prepare the data, especially ways to handle very sparse and large data, like using different filtering rules or ways to represent features. We also need to study more closely how features work together to affect performance and check how changing filtering or feature choices affects the results. Testing on more large datasets or different parts of them would also help.

Important next steps include finding ways to make the model faster and handle bigger data, maybe by using multiple computers at once or simpler versions of the model. Also, improving how we handle features is important. For example, using unique book codes like ISBNs (international standard numbers uniquely identifying a book) might simplify the features needed. We could also try adding more details (like full book info, information from review text using NLP, or from product pictures using computer vision) if we have enough computing power, to give the model better context.

Regarding specific problems to focus on, having to remove items with few interactions because of computer limits shows we need future research on finding and reducing unfairness (bias). Also, besides knowing

the total training time, looking closely at what parts take the longest could help speed things up.

Overall, this study looked closely at the problems of using ENSFM on large, real datasets. It shows the difference between how well it works on test data versus real-world challenges. Although we showed the model can run, the results make it clear that big improvements are needed in handling data, choosing features, making the model faster, and ensuring fairness. This study sets the foundation by pointing out these specific issues and prepares for future research focused on making advanced recommenders such as this thesis better and faster in real situations.

REFERENCES

Bayer, I., He, X., Kanagal, B., & Rendle, S. (2017). A Generic Coordinate Descent Framework for Learning from Implicit Feedback. *Proceedings of the 26th International Conference on World Wide Web (WWW '17)* (pp. 1341–1350). Geneva, Switzerland: International World Wide Web Conferences Steering Committee. Retrieved from https://doi.org/10.1145/3038912.3052694

Chen, C., Zhang, M., Ma, W., Liu, Y., & Ma, S. (2020). Efficient Non-Sampling Factorization Machines for Optimal Context-Aware Recommendation. *The Web Conference 2020 (WWW '20)*, (p. 11). Taipei, Taiwan. Retrieved from https://doi.org/10.1145/3366423.3380303

Chen, C., Zhang, M., Wang, C., Ma, W., Li, M., Liu, Y., & Ma, S. (2019). An Efficient Adaptive Transfer Neural Network for Social-aware Recommendation. *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'19)* (pp. 225–234). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/3331184.3331192

Chen, Y., Ren, P., Wang, Y., & de Rijke, M. (2019). Bayesian Personalized Feature Interaction Selection for Factorization Machines. *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'19)* (pp. 665–674). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/3331184.3331196

Deshpande, M., & Karypis, G. (2004). Item-based Top-N Recommendation Algorithms. *ACM Transactions on Information Systems (TOIS), 22*(1), 143–177. Retrieved from https://doi.org/10.1145/963770.963776

Developers, T. (2025, March 17). *TensorFlow*. Retrieved April 10, 2025, from GitHub: https://github.com/tensorflow/tensorflow/blob/master/README.md

He, X., & Chua, T.-S. (2017). Neural Factorization Machines for Sparse Predictive Analytics. *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '17)* (pp. 355–364). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/3077136.3080777

He, X., Chen, T., Kan, M.-Y., & Chen, X. (2015). TriRank: Review-aware Explainable Recommendation by Modeling Aspects. *Proceedings of the 24th ACM International Conference on Information and Knowledge Management (CIKM '15)* (pp. 1661–1670). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/2806416.2806504

He, X., He, Z., Song, J., Liu, Z., Jiang, Y.-G., & Chua, T.-S. (2018). NAIS: Neural Attentive Item Similarity Model for Recommendation. *IEEE Transactions on Knowledge and Data Engineering, 30*(12), 2354–2366. Retrieved from https://doi.org/10.1109/TKDE.2018.2831682

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural Collaborative Filtering. *Proceedings of the 26th International Conference on World Wide Web (WWW '17)* (pp. 173–182). Republic and Canton of Geneva, CHE: International World Wide Web Conferences Steering Committee. Retrieved from https://doi.org/10.1145/3038912.3052569

He, X., Zhang, H., Kan, M.-Y., & Chua, T.-S. (2016). Fast Matrix Factorization for Online Recommendation with Implicit Feedback. *Proceedings of the 39th International ACM SIGIR Conference on*

Research and Development in Information Retrieval (SIGIR '16) (pp. 549–558). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/2911451.2911489

Hou, Y., Li, J., He, Z., Yan, A., Chen, X., & McAuley, J. (2024). *Bridging Language and Items for Retrieval and Recommendation.* arXiv. Retrieved from https://arxiv.org/abs/2403.03952

Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. *IEEE International Conference on Data Mining (ICDM)* (pp. 263–272). IEEE. Retrieved from https://www.researchgate.net/publication/220765111_Collaborative_Filtering_for_Implicit_Feedback_Datasets

Inc., Y. (n.d.). *Yelp Open Dataset*. Retrieved April 11, 2025, from Yelp Data Licensing: https://business.yelp.com/data/resources/open-dataset/

Järvelin, K., & Kekäläinen, J. (2002). Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems (TOIS), 20*(4), 422–446. Retrieved from https://doi.org/10.1145/582415.582418

Jupyter, P. (n.d.). *Jupyter*. Retrieved April 9, 2025, from jupyter.org: https://jupyter.org/

Lab, M. (n.d.). *Amazon Reviews'23*. Retrieved April 11, 2025, from Amazon Reviews 2023: https://amazon-reviews-2023.github.io/

Le, H. (2019). *Recommender System for E-commerce Application.* Espoo, Finland. Retrieved from https://www.theseus.fi/bitstream/handle/10024/173892/Le_Hieu.pdf?sequence=2

Ma, H., Xie, R., Meng, L., Feng, F., Du, X., Sun, X., . . . Meng, X. (2024). *Negative Sampling in Recommendation: A Survey and Future Directions.* ACM. Retrieved from https://doi.org/10.48550/arXiv.2409.07237

Moreschini, M. (2021). *Bayesian Personalized Ranking Sampling Techniques.* Master Thesis, Milan, Italy. Retrieved from https://www.politesi.polimi.it/retrieve/47fe5e3e-a9c7-4c4e-bcd1-2af76cbaf88e/2021_12_MORESCHINI.pdf

Ning, X., & Karypis, G. (2011). SLIM: Sparse Linear Methods for Top-N Recommender Systems. *Proceedings of the 2011 IEEE 11th International Conference on Data Mining (ICDM 2011)* (pp. 497–506). IEEE. Retrieved from https://doi.org/10.1109/ICDM.2011.134

Pilászy, I., Zibriczky, D., & Tikk, D. (2010). Fast ALS-based Matrix Factorization for Explicit and Implicit Feedback Datasets. *Proceedings of the Fourth ACM Conference on Recommender Systems (RecSys '10)* (pp. 71–78). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/1864708.1864726

Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence (UAI '09)* (pp. 452–461). Arlington, Virginia, USA: AUAI Press. Retrieved from https://arxiv.org/pdf/1205.2618

Rendle, S., Gantner, Z., Freudenthaler, C., & Schmidt-Thieme, L. (2011). Fast Context-Aware Recommendations with Factorization Machines. *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '11)* (pp. 635–644). New York, NY, USA:

Association for Computing Machinery. Retrieved from https://doi.org/10.1145/2009916.2010002

Santana, I. A., & Domingues, M. A. (2020). *A Systematic Review on Context-Aware Recommender Systems Using Deep Learning and Embeddings.* arXiv. Retrieved from https://arxiv.org/pdf/2007.04782

Team, M. D. (n.d.). *Matplotlib*. Retrieved April 9, 2025, from matplotlib.org: https://matplotlib.org/

Team, P. D. (n.d.). *pandas*. Retrieved April 9, 2025, from pandas.pydata.org: https://pandas.pydata.org/

Xiao, J., Ye, H., He, X., Zhang, H., Wu, F., & Chua, T.-S. (2017). *Attentional Factorization Machines: Learning the Weight of Feature Interactions via Attention Networks.* Retrieved from https://doi.org/10.48550/arXiv.1708.04617

Xin, X., Chen, B., He, X., Wang, D., Ding, Y., & Jose, J. M. (2019). CFM: Convolutional Factorization Machines for Context-Aware Recommendation. *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI'19)* (pp. 3926–3932). AAAI Press. Retrieved from https://doi.org/10.24963/ijcai.2019/545

Xin, X., Yuan, F., He, X., & Jose, J. M. (2018). Batch IS NOT Heavy: Learning Word Representations From All Samples. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1853–1862). Melbourne, Australia: Association for Computational Linguistics. Retrieved from https://doi.org/10.18653/v1/P18-1172

Yuan, F., Guo, G., Jose, J. M., Chen, L., Yu, H., & Zhang, W. (2016). LambdaFM: Learning Optimal Ranking with Factorization Machines Using Lambda Surrogates. *Proceedings of the 25th ACM*

International Conference on Information and Knowledge Management (CIKM '16) (pp. 227–236). New York, NY, USA: Association for Computing Machinery. Retrieved from https://doi.org/10.1145/2983323.2983758

Yuan, F., Xin, X., He, X., Guo, G., Zhang, W., Chua, T.-S., & Jose, J. M. (2018). Learning Embeddings From Positive Unlabeled Data with BGD. *Conference on Uncertainty in Artificial Intelligence (UAI 2018).* Monterey, CA, USA. Retrieved from https://auai.org/uai2018/proceedings/papers/68.pdf

APPENDICES

Appendix 1. Statistical Overview of All the Datasets.

Table 7. Statistics of datasets in this thesis

Dataset	#User	#Item	#Features	#Interac-tion	#Field
Frappe	957	4,084	5,382	96,203	10
Last.fm	1,000	20,301	37,358	214,574	4
Movielens	6,040	3,706	10,021	1,000,209	6
Yelp2018	1,987,896	150,346	5,976,321	6,990,280	9
Multi-cate-gory of Yelp2018	21,114	50,673	75,495	277,417	11
Shopping of Yelp2018 multi-cate-gory	2,908	11,164	16,809	32,655	10
Food of Yelp2018 multi-cate-gory	10,684	18,204	32,524	121,963	10
Restaurant of Yelp2018	32,590	31,722	68,339	381,429	10

multi-category					
Amazon Books Reviews 2023	10,3M	4,4M	145,611,112	29,5M	10
Multi-category of Amazon2023	57,350	62,503	119,882	298,738	10
Arts & Photography of Amazon2023 multi-category	10,180	10,690	20,690	33,781	9
Genre Fiction of Amazon2023 multi-category	35,391	38,709	74,126	211,966	9
Children's Books of Amazon2023 multi-category	30,183	36,299	67,928	160,309	9

Appendix 2. Extract Unique JSON Keys from Yelp2018 Dataset.

```
import json

# Create sets to store unique keys
user_keys = set()
business_keys = set()
review_keys = set()
tip_keys = set()
checkin_keys = set()

# Paths to the JSON files
user_path = 'yelp_user.json'
business_path = 'yelp_business.json'
review_path = 'yelp_review.json'
tip_path = 'yelp_tip.json'
checkin_path = 'yelp_checkin.json'

# Function to extract keys from a JSON file
def extract_keys(file_path, key_set):
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if line:
                data = json.loads(line)
                key_set.update(data.keys())

# Extract keys from each file
extract_keys(user_path, user_keys)
extract_keys(business_path, business_keys)
extract_keys(review_path, review_keys)
extract_keys(tip_path, tip_keys)
extract_keys(checkin_path, checkin_keys)

# Print results
print("\n🔑 Keys in user JSON:")
for key in sorted(user_keys):
    print("-", key)

print("\n🔑 Keys in business JSON:")
for key in sorted(business_keys):
    print("-", key)

print("\n🔑 Keys in review JSON:")
for key in sorted(review_keys):
    print("-", key)

print("\n🔑 Keys in tip JSON:")
for key in sorted(tip_keys):
    print("-", key)
```

```
print("\n🔑 Keys in checkin JSON:")
for key in sorted(checkin_keys):
    print("-", key)
```

Appendix 3. Extract Unique JSONL Keys from Amazon Books Review 2023 Dataset.

For Books.jsonl file:

```
import json
import pandas as pd

# Path to the JSONL file
BOOKS_PATH = "Books.jsonl/Books.jsonl"

# === Load limited lines from JSONL ===
def load_meta_data(path, max_lines=1000):
    data = []
    with open(path, 'r', encoding='utf-8') as f:
        for i, line in enumerate(f):
            if i >= max_lines:
                break
            try:
                data.append(json.loads(line))
            except:
                continue
    return pd.DataFrame(data)

# === Load first 1000 rows safely ===
df_meta = load_meta_data(BOOKS_PATH, max_lines=1000)

# === Display structure ===
print("✅ Columns in Books.jsonl:")
print(df_meta.columns)
```

For meta_Books.jsonl file:

```
import json
import pandas as pd

# Path file meta_Books.jsonl
META_BOOKS_PATH = "meta_Books.jsonl/meta_Books.jsonl"

# === Load entire data from JSONL ===
def load_meta_data(path):
    data = []
    with open(path, 'r') as f:
        for line in f:
            try:
                data.append(json.loads(line))
            except:
                continue
    return pd.DataFrame(data)

# === Load data ===
df_meta = load_meta_data(META_BOOKS_PATH)

# === Check the structure of the data ===
print("Columns in meta_Books.jsonl:")
print(df_meta.columns)
```

Appendix 4. Listing the Book Category Distribution in the Amazon Books Reviews 2023 dataset.

```
import json
import numpy as np
import matplotlib.pyplot as plt
def                    analyze_category_frequency(file_path,
min_category_frequency):
    """
     Count the total number of occurrences of each category
from the meta book file and
      plot a bar chart for categories with at least
min_category_frequency occurrences,
     sorted in descending order. Also prints the summary
statistics.
    """
    category_counts = {}
    total_records = 0

    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            total_records += 1
            data = json.loads(line.strip())
             # Assumes the data includes a 'categories' key
containing a list of categories
            categories = data.get('categories')
            if categories and isinstance(categories, list):
                for category in categories:
                                  category_counts[category]  =
category_counts.get(category, 0) + 1

    print("=== Category Frequency Analysis Summary ===")
    print(f"Total records processed: {total_records}")
           print(f"Total    unique    categories    found:
{len(category_counts)}\n")

     # Filter out categories below the minimum frequency
threshold
       filtered_counts  =  {cat:  cnt  for  cat,  cnt  in
category_counts.items() if cnt >= min_category_frequency}

   print(f"Categories with at least {min_category_frequency}
occurrences: {len(filtered_counts)}")

    # Sort categories by total count in descending order
      sorted_categories  =  sorted(filtered_counts.items(),
key=lambda x: x[1], reverse=True)

    # Print detailed summary
    print("\nDetailed Category Counts:")
    for cat, cnt in sorted_categories:
        print(f"{cat}: {cnt}")
```

```
    # Prepare data for plotting
    categories = [cat for cat, cnt in sorted_categories]
    counts = [cnt for cat, cnt in sorted_categories]

    plt.figure(figsize=(12, 8))
    plt.bar(categories, counts, color='skyblue')
    plt.xlabel('Category')
    plt.ylabel('Total Occurrences')
       plt.title(f'Total  Occurrences  per  Category  (>=
{min_category_frequency} occurrences)')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()

    return filtered_counts

# Example usage in a Notebook
# Set file paths (adjust these to your local file locations
if necessary)
meta_books_file                                          =
"meta_Books.jsonl/meta_Books_processed.jsonl"
books_file = "Books.jsonl/Books_processed.jsonl"

# Set minimum category frequency threshold
min_category_frequency = 5000

# Call the functions and capture summaries
category_stats = analyze_category_frequency(meta_books_file,
min_category_frequency)
```

```
Detailed Category Counts:
Literature & Fiction: 711135
Children's Books: 421498
Arts & Photography: 205933
Genre Fiction: 203322
History: 182800
```